

SignSpeak AI

A Six-Chapter Thesis on Sign Language Recognition, Learning, and Deployment

*Prepared from the implemented system, trained checkpoints, and thesis model analysis
artifacts*

Project: SignSpeak AI

Date: May 17, 2026

Environment: FastAPI, React, PyTorch, PostgreSQL

Abstract

This thesis presents SignSpeak AI, a practical sign-language platform that combines isolated-word recognition, alphabet practice, live webcam inference, role-based learning workflows, and administrative oversight in a single deployable system. The project focuses on two landmark-based recognition pipelines. The first is a word-level recognizer trained on 80 American Sign Language vocabulary items represented as 32-frame MediaPipe landmark sequences with 167 features per frame. The second is an alphabet classifier trained on 26 letter classes for guided practice and live feedback. The implemented system replaces earlier RGB-heavy experimentation with lighter landmark models that are better aligned with real-time educational use.

The latest deployed word model, `word_landmarks_v14_ft`, was fine-tuned from v13 after expanding the candidate word dataset from 4759 to 4879 processed sequences. It reached 92.24% best validation accuracy and 91.97% held-out test accuracy across 735 test sequences. The alphabet benchmark, `alphabet_landmarks_v8`, achieved 96.24% test accuracy on 2,183 test samples. The thesis model analysis integrates checkpoint comparisons, training curves, per-class accuracy, weak-class improvement tracking, and dataset growth visualizations to show how targeted data expansion and warm-start fine-tuning improved the final system.

Beyond recognition models, the thesis documents the end-to-end application stack, including the student, teacher, and admin experiences; PostgreSQL-backed authentication; booking and profile management; and the new admin workspace for monitoring users, teachers, classes, and active model deployment. The result is a complete software artifact that supports both academic analysis and practical operation.

Table of Contents

1. Chapter One: Introduction
 2. Chapter Two: Background and Related Context
 3. Chapter Three: System Analysis and Design
 4. Chapter Four: Data Pipeline and Model Development
 5. Chapter Five: Experimental Results and Thesis Model Analysis
 6. Chapter Six: Conclusion and Future Work
- References

Chapter One: Introduction

SignSpeak AI was developed to address a practical gap between academic sign-language recognition experiments and deployable educational software. Many sign-language prototypes end at offline model evaluation, while many teaching applications stop at static content presentation without integrated recognition. This project bridges those concerns by combining machine-learning models, live inference services, and a role-based web platform in one codebase.

The project supports three main educational actors. Students practice alphabet handshapes, receive live predictions, book sessions, and track progress. Teachers manage profiles, classes, and learning interactions. Administrators monitor platform activity, inspect user and teacher records, review operational metrics, and supervise the active model deployment. This broad system scope is important because recognition quality alone does not determine product value; the surrounding workflow determines whether a trained model can be used consistently.

The central research objective of the thesis is to evaluate whether a landmark-based design can provide strong accuracy for isolated sign recognition while remaining efficient enough for a real-time educational platform. A secondary objective is to show how iterative dataset expansion, targeted remediation of weak classes, and fine-tuning from a prior checkpoint affect measurable model performance.

The thesis is guided by the following research questions:

1. Can MediaPipe landmark sequences provide a practical foundation for both word-level recognition and alphabet practice?
2. How much improvement is obtained by expanding the word dataset and warm-starting a new model from a stronger checkpoint?
3. How can model quality be integrated into a complete learning platform that includes authentication, profiles, classes, and administration?
4. Which word classes remain difficult even after substantial improvement, and what does that imply for future work?

The scope of the thesis is limited to isolated recognition rather than continuous sign-to-sentence translation. This constraint is deliberate. The isolated setting matches the available dataset scale, the platform objectives, and the latency requirements of webcam-based educational interaction.

Chapter Two: Background and Related Context

Sign-language recognition systems often balance three competing concerns: representation quality, data requirements, and inference speed. RGB video models can capture appearance-rich cues but are typically heavier, more sensitive to background conditions, and harder to align with fast interactive deployments. Landmark-based models replace much of the raw visual complexity with explicit geometric structure, making them attractive for hand-centric recognition tasks.

The SignSpeak AI project evolved through this trade-off. Early repository work included RGB-based recognition experiments, but the final platform emphasized MediaPipe landmarks because they reduce spatial redundancy and expose the motion patterns that dominate handshape-based classification. This shift also simplified live inference because the deployed backend could operate on structured hand trajectories rather than full image tensors.

Educational context also shapes the system design. Alphabet practice is not identical to word recognition. Alphabet letters are often static or short motion gestures with frequent handshape similarities, while isolated words can depend more strongly on temporal dynamics, movement direction, and transitional cues. As a result, the platform uses two specialized models rather than forcing a single architecture across all tasks.

The broader software context matters as well. A machine-learning model embedded inside a real product must coexist with authentication, database storage, profile editing, session management, and monitoring. This thesis therefore treats SignSpeak AI as a complete applied system instead of a narrow benchmark submission.

Key platform technologies include the following:

1. PyTorch for model definition, optimization, checkpointing, and evaluation.
2. MediaPipe landmarks for hand tracking and geometric sequence extraction.
3. FastAPI for backend inference endpoints, authentication, and admin APIs.
4. React for the client application used by students, teachers, and administrators.
5. PostgreSQL for persistent account and application data.

Chapter Three: System Analysis and Design

SignSpeak AI is organized as an integrated full-stack application. The backend exposes APIs for authentication, alphabet prediction, word prediction, teacher workflows, student workflows, and admin monitoring. The frontend consumes these APIs through a role-aware application shell. The data layer persists users, profiles, schedules, and related operational records. This architecture allows machine-learning outputs to be embedded inside structured user flows.

The student experience focuses on learning interaction. Students can log in, access alphabet practice, monitor live predictions, and participate in booking or teacher-driven learning flows. The teacher experience exposes profile information, classes, and instructional presence. The newly added admin experience provides operational visibility into system-wide activity and model deployment details, including active checkpoints, dataset counts, and performance summaries.

The admin workspace is especially relevant to model lifecycle management. By separating the admin dashboard from user and teacher navigation, the platform now supports direct review of users, teacher profiles, and classes through dedicated routes. This design reduces dashboard clutter and provides clearer ownership over operational tasks.

The system architecture can be summarized in four layers:

1. Capture layer: webcam or uploaded video provides raw sign input.
2. Representation layer: MediaPipe extracts normalized landmark sequences.
3. Inference layer: specialized PyTorch models classify words or alphabet letters.
4. Application layer: FastAPI and React present results inside learning and admin workflows.

For deployment, the current word service points to the checkpoint `artifacts/word_landmarks_v14_ft/best.pt`. Inference also uses mirrored test-time augmentation and sequence densification for short landmark dropouts, which improves runtime robustness without requiring retraining at prediction time.

Chapter Four: Data Pipeline and Model Development

The final recognition stack is landmark-first. Each word sample is converted into a fixed-length sequence of 32 frames. Each frame contains 167 features that encode tracked hand information. The processed candidate dataset grew from 4759 sequences in the v5 candidate build to 4879 sequences in the v6 candidate build, while the vocabulary remained fixed at 80 target words. This expansion specifically targeted classes that were weak in earlier checkpoints.

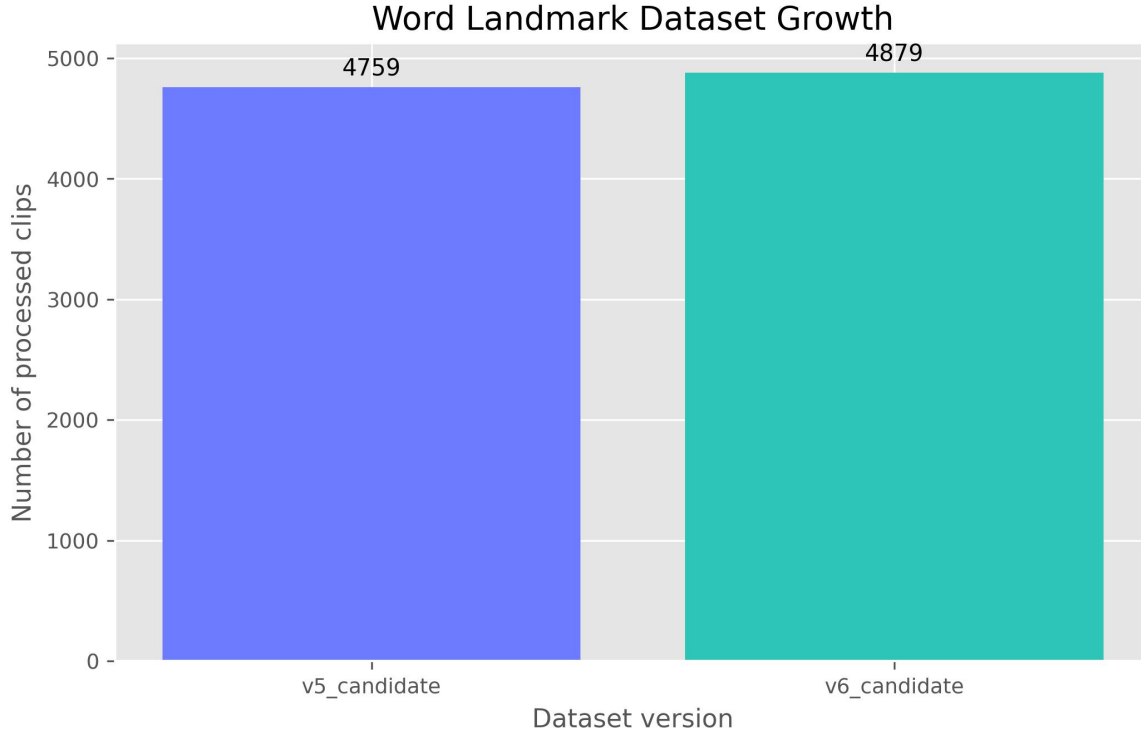


Figure 1. Growth of the processed word-landmark dataset across development stages.

The word recognizer is implemented as a bidirectional recurrent model with temporal attention. Input sequences are first normalized and projected into a higher-dimensional embedding space. A multi-layer bidirectional LSTM models temporal dependencies, and an additive attention module selects the most informative frames before final classification. The deployed v14_ft configuration uses a hidden dimension of 576, two recurrent layers, dropout of 0.4, and an attention dimension of 144.

Training uses AdamW optimization, cosine annealing of the learning rate, label smoothing, class weighting, and grouped data splitting to reduce leakage. The training pipeline was improved to fill dropped landmark frames, apply temporal cropping, and support warm-start initialization from a prior checkpoint. The v14_ft model was fine-tuned from v13 using the expanded candidate set rather than trained blindly from scratch, because the scratch run was less stable.

The alphabet model serves a different purpose and uses a lighter architecture. Instead of a recurrent attention pipeline, it uses a feed-forward multilayer perceptron over landmark-derived features, optimized for fast letter practice and live feedback. This separation of architectures reflects the difference between short static alphabet gestures and more temporally expressive word gestures.

Subsystem	Classes	Representation	Model	Best Checkpoint
Word recognition	80 words	32-frame landmark sequences	BiLSTM with temporal attention	word_landmarks_v14_ft
Alphabet practice	26 letters	landmark features	MLP classifier	alphabet_landmarks_v8

Chapter Five: Experimental Results and Thesis Model Analysis

This chapter presents the central thesis model analysis. The analysis is based on saved checkpoint artifacts, classification reports, and matplotlib visualizations generated directly from the repository. The focus is on three word checkpoints, v12, v13, and v14_ft, as well as the final alphabet benchmark.

Checkpoint	Epochs	Best Validation Accuracy	Test Accuracy	Test Loss
word_landmarks_v12	143	69.81%	67.78%	1.8516
word_landmarks_v13	102	74.06%	74.06%	1.5141
word_landmarks_v14_ft	44	92.24%	91.97%	0.7729

The progression from v12 to v14_ft shows the strongest quantitative result in the project. Test accuracy improved from 67.78% in v12 to 74.06% in v13 and then to 91.97% in v14_ft. The improvement from v12 to v14_ft is 24.19 percentage points. This gain was achieved without changing the vocabulary size, which means the increase came from better data coverage and a stronger optimization path.

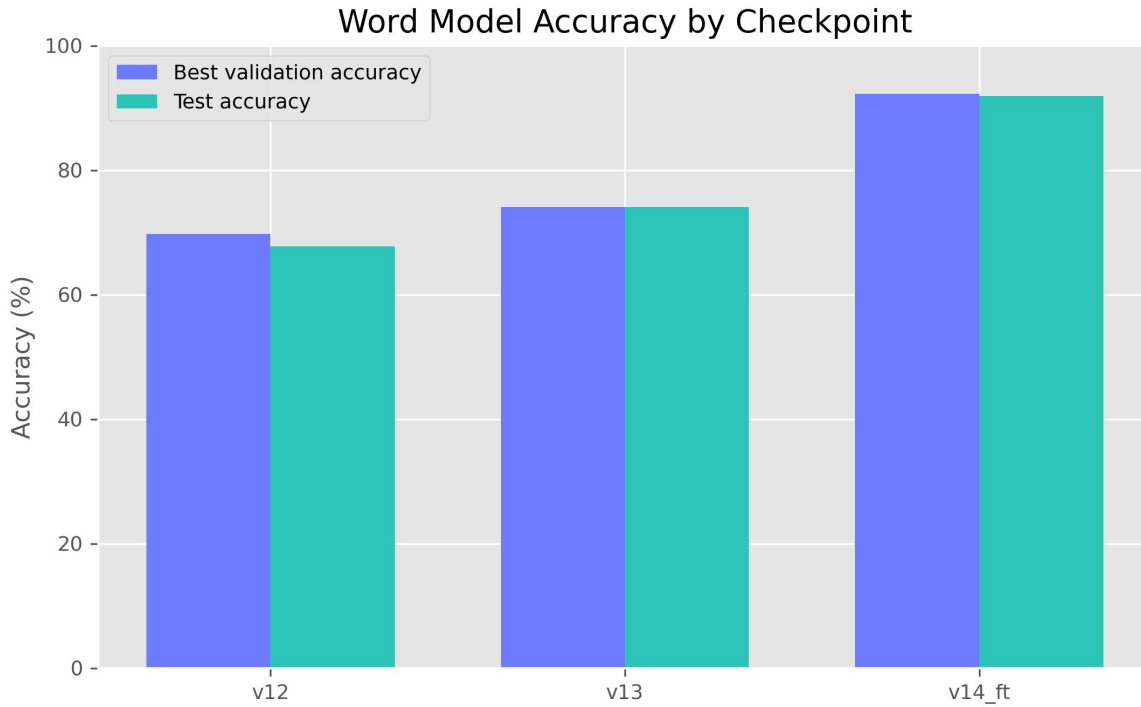


Figure 2. Comparison of held-out word-model accuracy across v12, v13, and v14_ft.

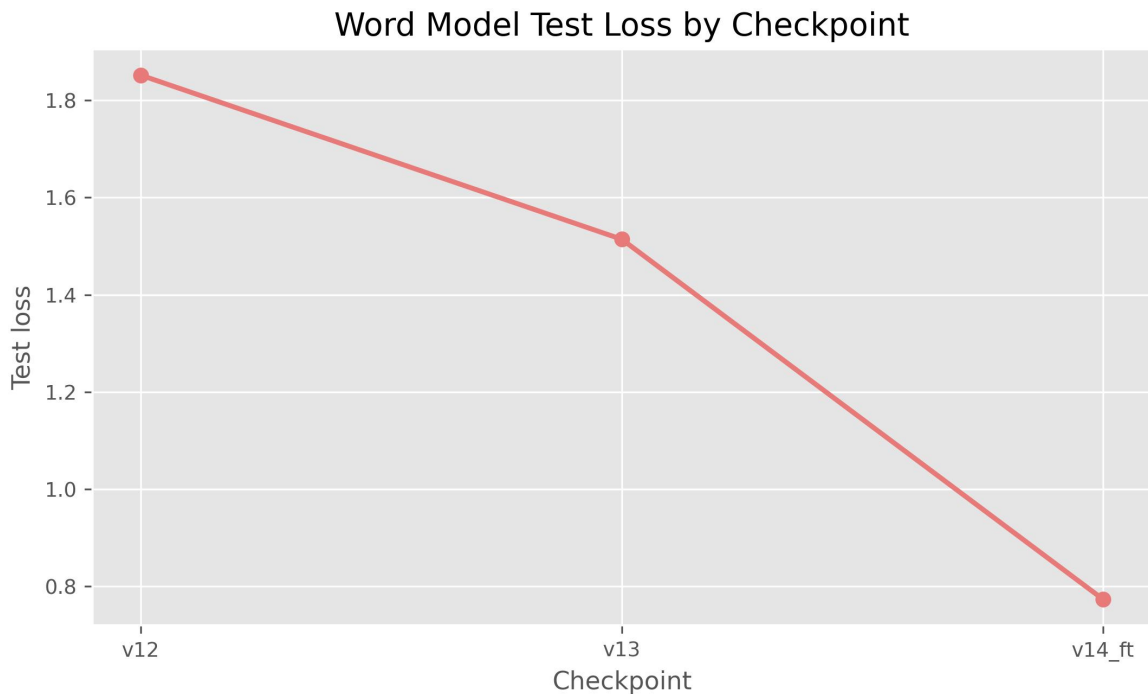


Figure 3. Comparison of held-out word-model test loss across checkpoints.

Training-curve analysis also supports the final model choice. The v14_ft history shows rapid convergence from a strong starting point, indicating that warm-start fine-tuning used prior knowledge efficiently. The best validation accuracy peaked at 92.24%, while the final held-out test accuracy remained closely aligned with validation behavior, suggesting good generalization rather than severe overfitting.

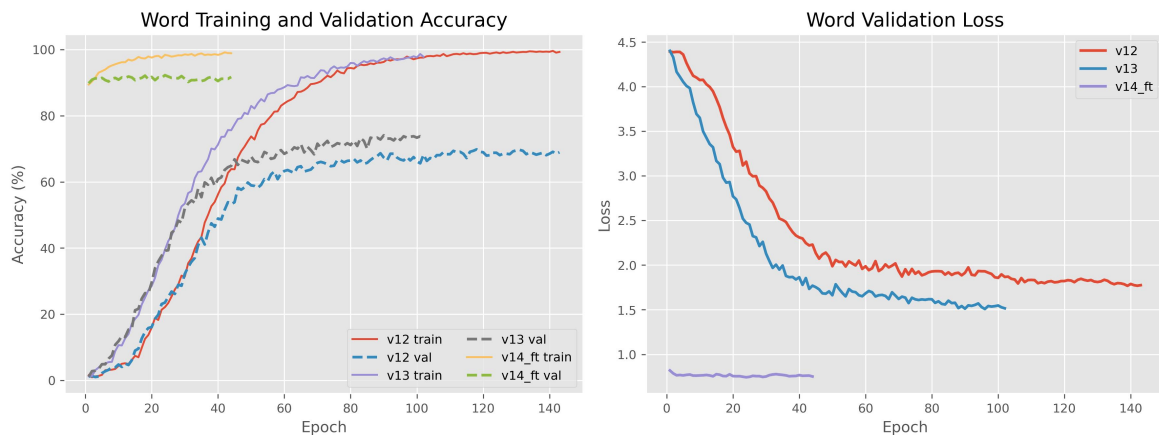


Figure 4. Training and validation behavior for the final word-model experiments.

Per-class analysis is especially important because aggregate accuracy can hide weak categories. The final v14_ft report shows that all 80 classes exceeded 50% recall on the held-

out test set, with many classes reaching perfect recall. This is a meaningful milestone because earlier checkpoints contained classes with severe failure cases.

Highest-Recall Word Classes	Recall	Lowest-Recall Word Classes	Recall
BABY	100%	READ	57%
BAD	100%	FLASHLIGHT	67%
BOOK	100%	COOL	75%
CANDY	100%	FINE	75%
CAT	100%	HOLD	75%
CLOUD	100%	TIME	75%
DEAF	100%	NIGHT	77%
DOG	100%	CRY	78%

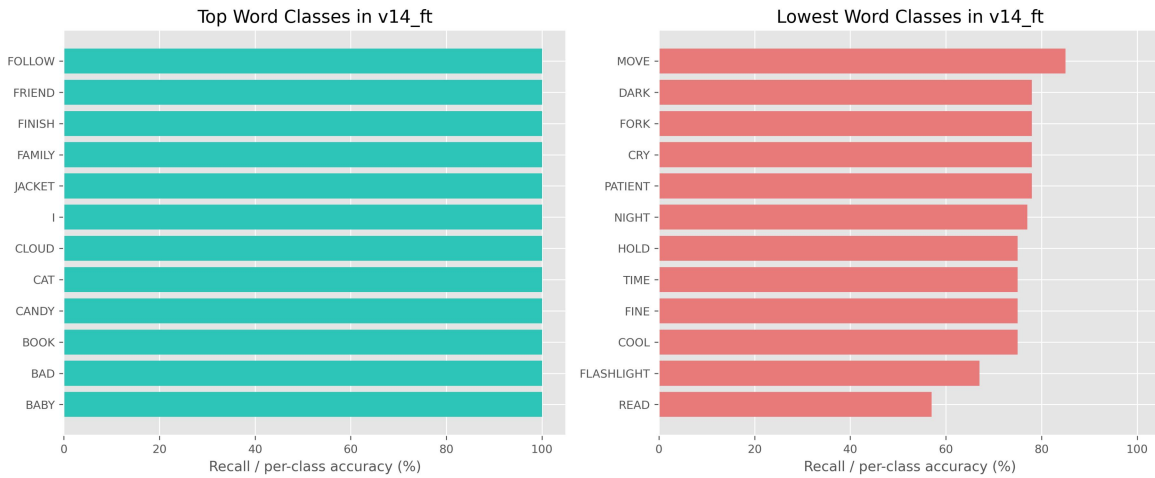


Figure 5. Top and bottom per-class recall for the final word model.

The weakest final word class was READ at 57% recall, followed by FLASHLIGHT at 67%, while classes such as BABY, BAD, CLOUD, FAMILY, LIVE, NAME, NO, and YOU reached perfect recall within the test split. The remaining difficult classes appear to be visually or temporally subtle gestures that may require additional motion diversity or better contextual cues.

Targeted weak-class remediation was successful. Earlier checkpoints struggled with labels such as ANIMAL, HOLD, LIVE, and PILLOW. After expanding the dataset and fine-tuning from v13, these classes improved substantially. This supports the practical strategy of error-driven data collection rather than indiscriminate scaling.

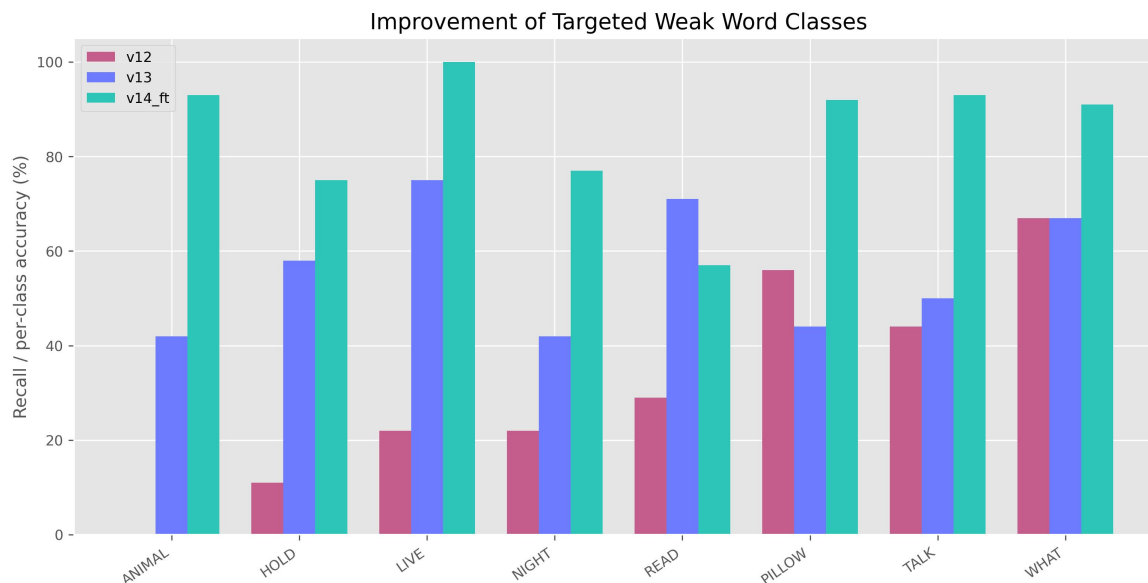


Figure 6. Improvement of selected weak word classes across checkpoint generations.

The alphabet benchmark remained strong throughout the later project stages. The final `alphabet_landmarks_v8` checkpoint reached 96.24% test accuracy over 2,183 samples, confirming that the educational practice component is accurate enough for live guided use.

Alphabet Checkpoint	Classes	Best Validation Accuracy	Test Accuracy	Test Loss
<code>alphabet_landmarks_v8</code>	26	95.14%	96.24%	0.3240

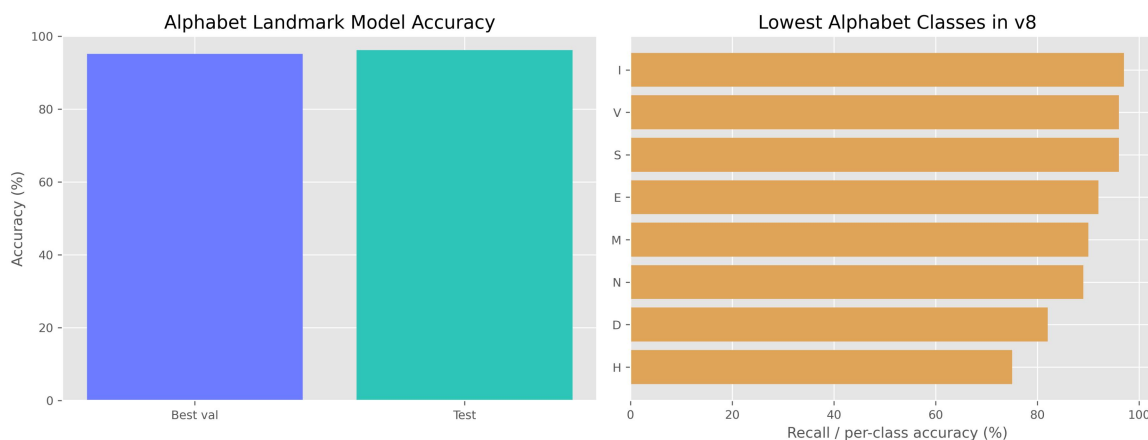


Figure 7. Summary of the final alphabet benchmark.

Most Challenging Alphabet Letters	Recall
H	75%

D	82%
N	89%
M	90%
E	92%

The most difficult alphabet letters in the final benchmark were H, D, N, M, and E, which is consistent with the known challenge of distinguishing similar handshapes. Even so, the lowest recall among these classes remained high enough for practical tutoring use.

Overall, the thesis model analysis demonstrates three key findings. First, landmark-based modeling is sufficient for strong educational sign-recognition performance in this scope. Second, targeted data acquisition combined with warm-start fine-tuning produced a clear measurable benefit. Third, performance reporting must include class-level analysis rather than only global accuracy, because model quality is unevenly distributed across vocabulary.

Chapter Six: Conclusion and Future Work

SignSpeak AI demonstrates that a landmark-first recognition pipeline can support a complete educational sign-language application. The final platform is not only a set of trained models but also a software system with authentication, role management, live webcam services, admin tooling, and reproducible model analysis. The strongest word checkpoint reached 91.97% test accuracy over 80 classes, while the alphabet benchmark reached 96.24%. These results are strong enough to justify the platform as a working applied system rather than a purely exploratory prototype.

The most important engineering lesson from the project is that architecture changes alone did not produce the final gain. The major improvement came from combining better data coverage, weak-class targeting, runtime robustness improvements, and warm-start fine-tuning. This reinforces a core applied-machine-learning principle: performance gains often come from tight iteration across data, training, and deployment rather than from a single isolated innovation.

Several future directions are justified by the current results:

1. Expand the word vocabulary beyond 80 labels with balanced support for each new class.
2. Add more difficult motion-rich or semantically similar signs to stress-test the model.
3. Introduce confusion-matrix driven review tools in the admin workspace for error triage.
4. Explore sentence-level or continuous-sign modeling once larger aligned corpora are available.
5. Evaluate cross-signer generalization under more diverse lighting, camera angle, and background conditions.

In conclusion, the thesis shows that SignSpeak AI is a credible end-to-end platform for isolated sign recognition and guided alphabet learning. The final system is academically analyzable, technically reproducible, and practically usable.

References

1. Project source code and training artifacts in the SignSpeak AI repository.
2. Metrics and classification reports from word_landmarks_v12, word_landmarks_v13, word_landmarks_v14_ft, and alphabet_landmarks_v8.
3. Dataset summaries from data/word_landmarks_v5_candidate/records.json and data/word_landmarks_v6_candidate/records.json.
4. Thesis analysis notebook and exported figures in docs/thesis_model_performance_analysis.ipynb and docs/thesis_figures/.
5. FastAPI, React, PyTorch, and MediaPipe software documentation used during implementation.